Lekcja 9. Wprowadzenie do programowania obiektowego - pytania i zadania

Klasa 4 technik programista | Programowanie aplikacji | INF.04

Zestaw sprawdza pamięć, rozumienie i zastosowanie pojęć związanych z paradygmatami programowania, programowaniem strukturalnym/proceduralnym oraz podstawami OOP.

A. Pytania kontrolne

1. Wyjaśnij pojęcie paradygmatu programowania.

2. Czym różni się programowanie imperatywne od deklaratywnego?

3. Co oznacza pojęcie stan programu?

4. Dlaczego instrukcja goto jest uznawana za niezalecaną w programowaniu strukturalnym?

5. Czym jest procedura lub funkcja w programowaniu proceduralnym?

6. Wyjaśnij, na czym polega podejście top-down.

7. Co oznacza dekompozycja problemu?

8. Wymień trzy podstawowe struktury sterujące w programowaniu strukturalnym.

9. Wyjaśnij różnicę między programowaniem proceduralnym a funkcyjnym.

10. Na czym polega główna idea programowania zorientowanego obiektowo?

11. Zdefiniuj pojęcia: klasa, obiekt, instancja.

12. Wymień cztery podstawowe cechy programowania obiektowego.

B. Prawda / fałsz

Zdanie	P/F
Paradygmat programowania jest konkretnym językiem programowania.	
W programowaniu imperatywnym kolejność instrukcji ma znaczenie.	
Programowanie proceduralne polega na dzieleniu problemu na funkcje lub procedury.	
Instrukcja goto ułatwia utrzymanie uporządkowanej struktury programu.	
Struktury wyboru służą do podejmowania decyzji w programie.	
Pętle należą do struktur iteracyjnych.	
Obiekt jest instancją klasy.	
Klasa jest zawsze gotowym obiektem zapisanym w pamięci operacyjnej.	
Hermetyzacja polega m.in. na łączeniu danych i operacji w jednej konstrukcji.	
Dziedziczenie, polimorfizm, hermetyzacja i abstrakcja należą do cech OOP.	

C. Uzupełnij tabelę

Pojęcie	Krótka definicja	Przykład / skojarzenie
paradygmat		
programowanie imperatywne		
programowanie proceduralne		
programowanie strukturalne		
programowanie obiektowe		
klasa		
obiekt		
hermetyzacja		
dziedziczenie		
polimorfizm		
abstrakcja		

D. Zadania praktyczne i analityczne

1. Analiza paradygmatu. Przeczytaj opis: „Program składa się z kolejnych instrukcji, które zmieniają wartości zmiennych aż do uzyskania wyniku”. Określ, jaki paradygmat opisano, i uzasadnij odpowiedź.

2. Dekompozycja. Rozbij problem „obsługa wypożyczalni rowerów” na co najmniej pięć funkcji. Dla każdej funkcji zapisz jej zadanie i przykładowe dane wejściowe/wyjściowe.

3. Struktury sterujące. Dla algorytmu obliczania średniej ocen wskaż, gdzie można zastosować sekwencję, wybór i iterację.

4. Refaktoryzacja myślenia. Masz program z jedną długą funkcją main(). Zaproponuj podział na funkcje i wyjaśnij, jak poprawi to czytelność oraz testowanie programu.

5. Model obiektowy. Dla systemu biblioteki zaproponuj trzy klasy. Dla każdej podaj co najmniej trzy pola i dwie metody.

6. Klasa a obiekt. Wyjaśnij na przykładzie klasy Samochod i obiektów auto1, auto2, czym różni się wzorzec od instancji.

7. Cechy OOP. Do każdego przykładu dopasuj cechę OOP: a) ukrycie pól przed bezpośrednią zmianą; b) klasa Uczen dziedziczy po klasie Osoba; c) różne klasy mają metodę wyswietl(); d) model zawiera tylko najważniejsze cechy obiektu.

8. Mini kod C++. Napisz szkic klasy Produkt z polami nazwa i cena oraz metodą wyswietl(). Następnie pokaż utworzenie jednego obiektu tej klasy.

E. Krótki sprawdzian - zastosowanie

1. Uzupełnij zdanie: Programowanie obiektowe wiąże ze sobą _____ oraz _____, tworząc konstrukcje nazywane klasami.

2. Zaznacz błędne stwierdzenie i popraw je: „Obiekt jest szablonem, a klasa jest konkretnym egzemplarzem utworzonym w pamięci”.

3. Napisz, dlaczego zagnieżdżanie struktur sterujących jest dopuszczalne, ale nadużywanie go może obniżyć czytelność programu.

4. Przyporządkuj pojęcia: sekwencja, wybór, iteracja do konstrukcji: for, if, instrukcje jedna po drugiej.

5. Podaj dwa powody, dla których dzielenie programu na funkcje ułatwia pracę programisty.

Kryteria sprawdzenia

- uczeń poprawnie wyjaśnia nowe pojęcia i skróty
- uczeń rozróżnia paradygmaty i umie podać przykłady
- uczeń wskazuje struktury sterujące i ich zastosowanie
- uczeń rozumie relację klasa - obiekt
- uczeń potrafi zaproponować proste klasy, pola i metody